# Certificate-driven polytope-hull contraction for interval branch and bound

Ignacio Araya
Escuela de Ingeniería Informática, Pontificia Universidad Católica de Valparaíso, Chile

Eduardo Pérez
Escuela de Ingeniería Informática, Pontificia Universidad Católica de Valparaíso, Chile

Victor Reyes
Escuela de Informática y Telecomunicaciones, Universidad Diego Portales, Chile

**[TODO:** *borrador del 2026-11-06*]

**Abstract**

Interval branch-and-bound solvers compute rigorous enclosures of the solutions of nonlinear systems and of global optima. Their polytope-hull contractor bounds each of the $n$ variables of a node over a linear relaxation by solving $2n$ small linear programs (LPs) with a floating-point LP solver, whose dual solutions are certified afterwards by the Neumaier–Shcherbina bound. That bound is valid for any multipliers, so any LP solver is safe to use; yet a general-purpose solver treats each of the $2n$ LPs as a new problem. We present a self-certifying simplex (SC) dedicated to the pass: it solves the $2n$ LPs in sequence on one basis, updating only what changes from one LP to the next. The certificate makes this simple: any basis yields a valid bound and can be made dual feasible, so SC never verifies, refactorizes before an optimum or runs a phase 1. A ladder of configurations in Ibex, from SoPlex as Ibex uses it to SC, attributes the gain to each step. On recorded passes, from the same boxes and relaxations, a pass of SC on relaxations of up to 30 rows costs 0.39–0.42 times that of SoPlex as Ibex uses it, and 20–25% less than the simplex of SoPlex driven directly, with the same contraction. Inside the search, the optimizer takes about 20% less CPU time than with SoPlex as Ibex uses it. On larger relaxations SC is slower, and a hybrid that chooses the engine by the size of the relaxation keeps the best of both.

## 1 Introduction

Interval branch-and-bound solvers compute, with mathematical certainty, an enclosure of all solutions of a system of nonlinear equations and inequalities, or the global optimum of a constrained problem [20]. Their efficiency depends on *contractors*, operators that shrink the current box without losing solutions [7]. Among them, contractors based on linear relaxations are among the most effective: the constraints are linearized over the box, by convex interval Taylor forms [3] or affine arithmetic [23, 2], and each variable is minimized and maximized over the resulting polytope. In Ibex this is the polytope-hull contractor, the linear part of X-Newton, which we introduced in [3]; we later combined the affine and Taylor-based relaxations [2].

Each call solves up to $2n$ linear programs, and the rigor of the result cannot come from the LP solver, which works in floating point. Current implementations therefore treat the LP solver as a black box and certify its output a posteriori: the dual solution is turned into a safe bound by

the Neumaier–Shcherbina procedure [21], and an infeasibility claim is accepted only if the returned Farkas ray passes the same verification. This design has a cost that is easy to miss: the $2n$ LPs of a node are small and closely related, typically two or three pivots apart, and each is a separate call to a general-purpose solver, which prepares the LP as a new problem, rebuilding its vectors, and refactorizes the basis and checks its solution before declaring an optimum; on these LPs that work is a large part of the cost of a call (Section 5).

The certificate already makes any LP solver safe to use: with the relaxation written in a homogeneous form, with one interval-bounded slack per row, *any* multiplier vector yields a valid bound on the box and on every sub-box, whatever solver produced it. A black box cannot take advantage of this, because it treats each call as a new problem. We design the simplex for the pass instead: the relaxation is loaded once, and the $2n$ LPs are solved in sequence on one basis, keeping the factorization and updating only what the new objective and the contracted box change. The certificate is what makes such a simplex simple. Its multipliers never need to be correct, so it performs no verification of its own and does not refactorize before declaring an optimum; and every basis can be made dual feasible by relocating its nonbasic variables, so no LP of the pass, and no LP after the box contracts, needs a phase 1 (the search for a feasible starting basis). We call this simplex *self-certifying* (SC). Against the simplex of SoPlex driven directly, which also keeps its basis from one LP to the next, we find that what SC saves is mostly the preparation of each LP as a new problem: switching off the refactorization and the checks of SoPlex recovers little. How much it is worth depends on the size of the relaxation and on what else the contractor does, and measuring that is part of the contribution.

**Positioning.** The certificate itself is not new: it is the safe bound of Neumaier and Shcherbina [21], already applied to filtering with linear relaxations [9, 24]. In those approaches the multipliers are a by-product of an external solver and certification is a post-processing step. Resuming a dual simplex inside an interval method goes back to Yamamura and coworkers [27, 28], for an LP emptiness test; SC extends that warm start to the $2n$ bound problems, with rigorous certification. Section 2 gives the details.

**Contributions.** The paper makes three contributions.

- *A simplex dedicated to the pass.* SC solves the $2n$ problems of a node in sequence on one basis, without preparing each as a new problem, without a phase 1, and without the verification and the refactorization before an optimum of a general-purpose solver (Section 4). The properties it relies on, validity of the certificate for any multipliers, exactness at an optimum and dual feasibility for free, follow from the Neumaier–Shcherbina bound and LP duality; we state them in Section 3.1 because the design rests on them, not as new results.

- *An attribution of the gain.* A ladder of configurations inside the same contractor of Ibex, from SoPlex as Ibex uses it, through the merging of twin rows (a simple preprocessing that halves the relaxation of square systems for every engine), a tuned SoPlex and its simplex driven directly, to SC and a hybrid, measures what each step contributes, separately on optimization problems and systems of equations and on small and large relaxations: on a bank of recorded passes, where every rung starts from the same boxes and relaxations, and inside the search, with one instance per family (Section 5).

- *The boundary of the approach.* SC pays on small relaxations, where the fixed cost of a call dominates (per pass, up to about 30 rows); beyond, the pivots dominate, and SoPlex makes

them cheaper by switching to a row representation of the basis that SC does not have. We measure that explanation, and give a hybrid contractor that chooses the engine per node by the size of the relaxation (Sections 5 and 6).

## 2 Background and related work

**Interval solvers and linear relaxations.** Interval branch and bound alternates bisection with contraction. Ibex, on which we build, composes constraint propagation (HC4 [6]), shaving-based consistencies such as ACID [22], the interval Newton operator for square systems [19], and contractors based on linear relaxations [7], whose relaxation comes from convex interval Taylor forms [3], from affine arithmetic [23], or from both [2]. QUAD [17] and GloptLab [9] use linear relaxations in the same way.

**Rigorous bounds for linear programs.** A floating-point LP solver returns approximate solutions, and two lines of work make its results rigorous. The first certifies them a posteriori: any approximate dual vector yields a safe bound once the residual is evaluated in interval arithmetic, and an approximate Farkas ray proves infeasibility under the same verification [21, 14, 15]. The second solves the LP exactly, in rational arithmetic [1], or to arbitrary precision by iterative refinement [12], as in exact mixed-integer programming [8]. Both treat the LP solver as given; we keep the a posteriori certificate and change the solver. Domes and Neumaier [9] applied the a posteriori bound to filtering with linear relaxations; in previous work [24] we wrote the relaxation in the homogeneous form used here and showed that the dual solutions of the bound LPs give the optimal preconditioner for an interval Gauss–Seidel projection, and Gleixner et al. [11] reuse the equivalent inequalities in sub-boxes. In all of them certification is a post-processing step, whose cost leads to applying it to a few promising variables [9] or only periodically [24]. Minimizing and maximizing each variable over a relaxation is also the optimization-based bound tightening of global optimization solvers, where rigor is not required [25, 5, 11].

**Warm starts in interval methods.** Yamamura and coworkers [27, 28] observed that the optimal tableau of the parent remains dual feasible after a bisection, and resume an LP emptiness test by the dual simplex method. The heuristic of skipping the bounds already attained by a primal point, used by the polytope-hull contractor of Ibex, comes from Baharev, Achterberg and Rév [4].

**Simplex techniques.** SC uses standard ingredients, described by Koberstein [16] for the dual method: the bounded-variable dual simplex with Harris' ratio test [13], and the revised method with a sparse LU factorization updated by the method of Forrest and Tomlin [10], as in SoPlex [26]. What is specific to our setting is their use for $2n$ closely related LPs over the same constraints, with a certificate attached to every basis.

## 3 The contractor

This section describes the contractor independently of the engine that solves its LPs: the relaxation and its certificate (Section 3.1), the pass over the bounds (Section 3.2) and the merging of twin rows (Section 3.3). Section 4 then describes SC, and Section 5 compares it with external LP solvers inside this same contractor. Only SC is new; the rest is the common ground on which the engines are compared.

### 3.1 The homogeneous relaxation and its certificate

Let $\mathbf{x} \in \mathbb{IR}^n$ be the current box. A linearizer returns a matrix $A \in \mathbb{R}^{m \times n}$ of floating-point numbers and an interval vector $\mathbf{b} \in \mathbb{IR}^m$, possibly with infinite bounds, such that

$$x \in \mathbf{x} \text{ is a solution } \implies Ax \in \mathbf{b}. \tag{1}$$

We use the two linearizers of our previous work [2]: the *affine* relaxation [23], and the *combined* relaxation (`compo`), which adds to the affine rows those of X-Newton [3], one per constraint and corner of the box. We introduce one slack per row, $s = Ax$, and write the relaxation in the homogeneous form

$$\bar{A}z = 0, \qquad z = (x, s) \in \mathbf{z} := \mathbf{x} \times \mathbf{s}, \qquad \bar{A} := [\, A \;\; -I_m \,], \tag{2}$$

where $\mathbf{s}_i := \mathbf{b}_i \cap \square(A_{i:}\,\mathbf{x})$ and $\square(\cdot)$ denotes evaluation in interval arithmetic. We call $P(\mathbf{x}) := \{z \in \mathbf{z} : \bar{A}z = 0\}$ the *relaxation polytope*.

**Lemma 1.** *If $x^* \in \mathbf{x}$ is a solution, then $(x^*, Ax^*) \in P(\mathbf{x})$.*

*Proof.* $A_{i:}x^* \in \mathbf{b}_i$ by (1) and $A_{i:}x^* \in \square(A_{i:}\mathbf{x})$ by the inclusion property of interval arithmetic; the equation holds by construction of $s$. □

The certificate combines the $m$ rows of $\bar{A}z = 0$ linearly, with one *multiplier* per row, into a single equation $\gamma^T z = 0$ that every point of the polytope satisfies. Any vector of multipliers $\lambda \in \mathbb{R}^m$ gives a valid equation; the best ones for bounding $z_k$ are the dual solutions of the LP that minimizes or maximizes $z_k$ over the polytope, and the simplex of Section 4 produces them.

**Definition 2** (Certificate)**.** For a vector of multipliers $\lambda \in \mathbb{R}^m$, let $\gamma \in \mathbb{IR}^{n+m}$ be an interval vector that contains $\gamma := \bar{A}^T\lambda$, computed in outward-rounded interval arithmetic. For $k \in \{1, \ldots, n+m\}$ with $0 \notin \gamma_k$, the *certified enclosure* of $z_k$ is

$$\mathbf{C}_k(\lambda, \mathbf{z}) := -\Big(\textstyle\sum_{j \neq k} \gamma_j\, \mathbf{z}_j\Big) \Big/ \gamma_k,$$

and $\mathbf{C}_k(\lambda, \mathbf{z}) := \mathbb{R}$ when $0 \in \gamma_k$; and the *residual* is $\mathbf{R}(\lambda, \mathbf{z}) := \sum_j \gamma_j\, \mathbf{z}_j$.

The idea is a single line. Every point $z$ of the polytope satisfies $\bar{A}z = 0$, hence $\gamma^T z = \lambda^T \bar{A} z = 0$ for any $\lambda$, that is, $\gamma_k z_k = -\sum_{j \neq k} \gamma_j z_j$. Bounding the right-hand side over the box in interval arithmetic, and dividing by $\gamma_k$, encloses $z_k$; and if the residual $\mathbf{R}$ excludes zero, no point of the box can satisfy $\gamma^T z = 0$, so the polytope is empty. A contraction step therefore replaces $\mathbf{z}_k$ by $\mathbf{z}_k \cap \mathbf{C}_k(\lambda, \mathbf{z})$, and an emptiness test checks $0 \notin \mathbf{R}(\lambda, \mathbf{z})$. All statements below hold for the floating-point $A$ returned by the linearizer; rounding in $\gamma$ and $\mathbf{C}_k$ is covered by outward rounding.

**Proposition 3** (Validity)**.** *Let $\lambda \in \mathbb{R}^m$ be arbitrary. For every $z \in P(\mathbf{x})$ and every $k$ with $0 \notin \gamma_k$, $z_k \in \mathbf{C}_k(\lambda, \mathbf{z})$; if $0 \notin \mathbf{R}(\lambda, \mathbf{z})$, then $P(\mathbf{x}) = \emptyset$ and $\mathbf{x}$ contains no solution. Both claims hold on every sub-box $\mathbf{x}' \subseteq \mathbf{x}$ with the matrix $\bar{A}$ computed on $\mathbf{x}$.*

*Proof.* For $z \in P(\mathbf{x})$ and the exact $\gamma = \bar{A}^T\lambda \in \gamma$, $\gamma^T z = \lambda^T \bar{A} z = 0$; both claims follow from the inclusion property of interval arithmetic and Lemma 1. The linearization on $\mathbf{x}$ remains valid on $\mathbf{x}'$, so the polytope over $\mathbf{x}'$ lies in $P(\mathbf{x})$. □

No property of $\lambda$ is used: it need not be optimal, feasible or accurate, and errors in it can only weaken the enclosure. Written back in the variables $x$, $\inf \mathbf{C}_k(\lambda, \mathbf{z})$ is the Neumaier–Shcherbina safe lower bound of $\min x_k$ subject to $Ax \in \mathbf{b}$, $x \in \mathbf{x}$, with approximate dual $\lambda$ [21], and the row-wise solution of the preconditioned system of Domes and Neumaier [9, §5]; validity on sub-boxes is used by Gleixner et al. [11] and in our earlier work [24]. The homogeneous form, introduced in [24], adds one detail: each slack is bounded by $\square(A_{i:}\mathbf{x})$ even when a side of $\mathbf{b}_i$ is infinite, so a multiplier of the "wrong" sign on a one-sided row gives a weaker but finite bound, and literally any $\lambda$ certifies. Two standard facts complete what the design uses. At an optimal basis of $\mathrm{LP}_k^{\min}$ with $z_k$ basic, $\inf \mathbf{C}_k(\lambda^*, \mathbf{z})$ equals the optimal value in exact arithmetic, by LP duality [24, Prop. 1]; in floating point the excess is at most $\sum_j \mathrm{wid}(\gamma_j)\,\mathrm{mag}(\mathbf{z}_j)$ plus the effect of the inaccuracy of $\lambda$, as for the Neumaier–Shcherbina bound. And every basis can be made dual feasible for free [16]:

**Proposition 4** (Dual-feasible start)**.** *The slack basis $B = -I_m$ is dual feasible for the bound problem of every structural variable $z_k$, $k \le n$, with $\mathbf{z}_k$ bounded. If every $\mathbf{z}_j$ is bounded, any basis becomes dual feasible after placing each nonbasic $z_j$ at its lower bound if its reduced cost $d_j > 0$ and at its upper bound if $d_j < 0$.*

*Proof.* With the slack basis the cost of every basic variable is zero, so $\lambda = 0$ and $d = e_k$, and $z_k$ at its lower bound (at its upper bound for the maximization) has the right sign. In general $d$ does not depend on where the nonbasic variables sit, and the relocation enforces the sign conditions whenever the required bound is finite. □

When the objective changes from one bound to the next, or the box contracts, relocation thus restores dual feasibility without a phase 1, which is what lets one basis serve the $2n$ problems; the case of a box change is the warm start of Yamamura and coworkers [27, 28]. Section 4 explains how the simplex produces $\lambda$.

**Why not a rigorous simplex.** Since any multipliers give a valid bound, rigor is needed only in the final evaluation of $\gamma$ and $\mathbf{C}_k$, not in the pivoting. Pivoting in interval arithmetic pays the width growth of interval Gaussian elimination [19], and solving each LP exactly or to high precision [1, 12] buys an exactness that the bound does not need; an early version of our contractor that pivoted in interval arithmetic was 2.8 times slower, with the same contraction**[TODO:** *decir sobre qué instancias se midió*].

### 3.2 One pass over the bounds

At each node the relaxation is computed, written in the form (2), and loaded once. The contractor then makes one pass over the up to $2n$ bound problems

$$\mathrm{LP}_k^{\sigma}: \quad \sigma\, z_k \quad \text{s.t.} \quad \bar{A}z = 0,\ z \in \mathbf{z}, \qquad k \le n,\ \sigma \in \{\min, \max\}, \tag{3}$$

and we write $\mathrm{LP}_k$ when the sense does not matter. Algorithm 1 gives the pass. The set $L$ holds the bounds still to solve (line 1). Each LP is warm-started from the final basis of the previous one, and the engine returns the multipliers of the basis where it stops (line 4). They certify the bound once, and the box is contracted before the next LP starts (line 6), so every later LP works on the smaller box. If the residual $\mathbf{R}(\lambda, \mathbf{z})$ excludes zero, the node is empty and the pass stops (line 5).

No other outcome stops the pass, because any multipliers certify (Proposition 3): an LP that stops at its iteration limit, or reports an infeasibility whose ray fails the residual test, still contracts with the multipliers it returns, if less than an optimal basis would. The polytope-hull contractor

**Algorithm 1** One pass over the bounds of a node. SOLVE is the LP engine, SC (Algorithm 2) or an external solver.

**Require:** relaxation $\bar{A}z = 0$ with twin rows merged, box $\mathbf{z}$
**Ensure:** contracted box $\mathbf{z}$, or $\emptyset$
1: $L \leftarrow \{(k, \sigma) : k \le n,\ \sigma \in \{\min, \max\}\}$
2: **while** $L \neq \emptyset$ **do**
3: $\quad (k, \sigma) \leftarrow$ next bound of $L$; remove it from $L$ ▷ skipping and order
4: $\quad \lambda \leftarrow \text{SOLVE}(\text{LP}_k^\sigma, \mathbf{z})$ ▷ warm start, at most 100 iterations
5: $\quad$ **if** $0 \notin \mathbf{R}(\lambda, \mathbf{z})$ **then return** $\emptyset$ ▷ Proposition 3
6: $\quad \mathbf{z}_k \leftarrow \mathbf{z}_k \cap \mathbf{C}_k(\lambda, \mathbf{z})$ ▷ Neumaier–Shcherbina bound [21]
7: **return** $\mathbf{z}$

of Ibex 2.8.9 instead abandons the pass in both cases; we corrected it, the correction has been submitted to the Ibex distribution, and every configuration of Section 5 uses it, including the one that stands for Ibex. Ibex limits SoPlex to 100 iterations per LP, and every engine of Section 5, SC included, uses that limit.

Line 3 chooses the next bound with two rules of the polytope-hull contractor of Ibex [4], which use the primal point of the last LP solved to optimality. A bound already attained by such a point is *skipped*: it is removed from $L$ without solving its LP, which would return that value, and the pass ends when every remaining bound is skipped. Among the others, the next one is the bound nearest to the last primal point, whose optimum lies closest to the vertex where the previous LP ended, so that it needs fewer pivots.

### 3.3 Merging twin rows

Rows of the relaxation that are proportional, within a relative tolerance of $10^{-9}$, are merged into one row whose slack interval is the intersection of their scaled intervals; the polytope does not change. The affine linearizer writes each equation as two one-sided rows with the same normal, so merging halves $m$ on square systems, and `compo` writes some rows twice, with opposite signs, far apart in the matrix. Merging is done once per node, before any LP, and gives a smaller LP to SC and to an external solver alike.

## 4 A simplex for the $2n$ bound problems

For each structural variable $x_k$ the pass of Section 3.2 needs the two bounds of $\text{LP}_k$ (3). A basis is a set of $m$ columns of $\bar{A}$ forming a nonsingular matrix $B$; with cost vector $c = \pm e_k$, the multipliers are $\lambda^T = c_B^T B^{-1}$ and the reduced costs $d = c - \bar{A}^T\lambda$. For the bounded-variable simplex, a basis with each nonbasic variable at one of its bounds is *dual feasible* if $d_j \ge 0$ for nonbasic variables at their lower bound and $d_j \le 0$ for those at their upper bound, and *primal feasible* if the basic variables lie within their bounds. This section describes how the simplex solves the $2n$ problems of a node; the design choices are evaluated in Section 5.

**A simplex for one pass.** SC is not a general LP solver. It solves the $2n$ LPs of one pass over a relaxation loaded once, in sequence and on one basis: between two LPs it keeps its factorization and recomputes only what the new objective and the contracted box change. The certificate is what makes this simple. Since it is valid for any $\lambda$ (Proposition 3), the simplex never has

to return a correct solution, only a good basis. A general-purpose LP solver, before declaring an optimum, refactorizes the basis to remove the error accumulated by the updates of its factorization, recomputes its vectors and verifies the solution, and the contractor then verifies it once more through the certificate. SC uses the final basis as it is: an inaccurate $\lambda$ yields a weaker bound, never an invalid one, so the error of the factorization is controlled only as far as pivoting requires. Likewise, since every basis can be made dual feasible (Proposition 4), no phase 1 is needed when the objective changes or the box contracts, and the $2n$ problems share one basis; with the primal warm start of Section 4.1 none is needed either, because the previous optimal basis stays primal feasible. Section 5.1 measures what this design is worth: against the simplex of SoPlex driven directly, the gain comes mostly from not preparing each LP as a new problem, more than from the omitted checks.

Algorithm 2 shows one call of SC, the SOLVE of Algorithm 1. Lines 2–8 prepare the LP from the state left by the previous one (Section 4.1). The loop of lines 9–23 combines both methods: it takes a dual step while some basic variable violates its bounds, a primal step while some reduced cost has the wrong sign, and stops at an optimal basis (line 19), at a Farkas ray (line 13) or at the iteration limit (line 23); Section 4.2 describes the steps. After a dual warm start the basis is dual feasible and only dual steps occur; after a primal one it is primal feasible and only primal steps occur. The two lines marked $\star$ are where the design described above departs from a general-purpose solver: line 4, the warm start, starts each LP from the final basis of the previous one, with its factors, and line 19 returns the basis as it is. The certificate is computed by the caller, in lines 5–6 of Algorithm 1 (Section 4.3).

### 4.1 Starting and warm-starting

The first LP of a pass starts from the slack basis $B = -I_m$ (line 2 of Algorithm 2), which is dual feasible for every $\mathrm{LP}_k$ (Proposition 4), and is solved by the dual method, with no phase 1: the costs of the slacks are zero, so $\lambda = 0$ and $d = c$ (line 5), and line 7 puts $z_k$ at its lower bound for a minimization and at its upper bound for a maximization. Between two consecutive LPs the objective changes and the contracted bound changes the box. SC keeps the basis and its factors (line 4) and recomputes $\lambda$ and $d$ for the new objective (line 5); two warm starts are then possible. The *dual* one relocates every nonbasic variable to the bound indicated by the sign of its new reduced cost (line 7), which restores dual feasibility, and continues with the dual method; the *primal* one keeps the nonbasic variables where they are and reoptimizes with primal steps, as an LP solver does. Under the dual method the certificate is available at every iteration, and it never gets worse:

**Proposition 5** (Monotone certified bound)**.** *Run the dual simplex method on* $\mathrm{LP}_k^{\min}$ *from a dual-feasible basis. At every iteration* $t$ *in which* $z_k$ *is basic,* $\inf \mathbf{C}_k(\lambda_t, \mathbf{z})$ *is a valid lower bound on* $z_k$ *over* $P(\mathbf{x})$ *and, in exact arithmetic, it is nondecreasing in* $t$*.*

*Proof.* Validity is Proposition 3. With $z_k$ basic, $d = e_k - \bar{A}^T\lambda_t$ has $d_k = 0$, so $\gamma_k = 1$, $\gamma_j = -d_j$ for $j \neq k$, and $\inf \mathbf{C}_k = \sum_{j\neq k} \min_{z_j \in \mathbf{z}_j} d_j z_j = \min_{z\in\mathbf{z}} d^T z$, the Lagrangian dual function of $\mathrm{LP}_k$ at $\lambda_t$, which the iterates of the dual simplex method do not decrease. □

Both warm starts reach an optimal basis and give the same bound; they differ in cost. A nonbasic variable that must move to its opposite bound costs the primal warm start an iteration, while relocation moves it for free but forces the basic values to be recomputed once per LP. On systems of equations every row is a narrow slab (its slack interval covers 13% of the range of the row over the box, on average), and a median of 1.9 nonbasic variables per LP change side, against 0.3 on optimization problems, whose slack intervals cover 48% of the range[**TODO:** *decir dónde se*

**Algorithm 2** SOLVE in SC: one LP of the pass. The relaxation is loaded once per node; the basis, its LU factors, $x_B$, $\lambda$ and $d$ persist from one call to the next. Comments marked $\star$ point to the work a general-purpose solver does differently.

**Require:** bound $(k, \sigma)$, so $c = \pm e_k$; box $\mathbf{z}$, contracted since the previous call

1: **if** first LP of the node **then**
2: $B \leftarrow -I_m$, factorized ▷ dual feasible for every $\mathrm{LP}_k$ (Prop. 4)
3: **else**
4: $B \leftarrow$ final basis of the previous LP, with its LU factors and $x_B$ ▷ $\star$ a solver starts anew
5: $\lambda \leftarrow B^{-T} c_B$; $d \leftarrow c - \bar{A}^T \lambda$ ▷ multipliers and reduced costs for $c$
6: **if** first LP **or** dual warm start **then**
7: place each nonbasic $z_j$ at its lower bound if $d_j \geq 0$, at its upper bound otherwise
8: $x_B \leftarrow B^{-1}(-\bar{A}_N z_N)$ ▷ values of the basic variables
9: **for** $t = 1, \ldots, 100$ **do**
10: **if** some basic variable violates its bounds **then** ▷ dual step
11: $r \leftarrow$ the basic variable with the largest violation; $\rho_r \leftarrow B^{-T} e_r$; $\alpha_r^T \leftarrow \rho_r^T \bar{A}$
12: $q \leftarrow$ ratio test on $d_j / \alpha_{rj}$ with Harris' rule
13: **if** no $q$ exists **then return** $\lambda \leftarrow \rho_r$ ▷ Farkas ray
14: $\alpha_q \leftarrow B^{-1} \bar{A}_{:q}$
15: **else if** some nonbasic $d_j$ has the wrong sign **then** ▷ primal step
16: $q \leftarrow$ the nonbasic variable with the largest such $|d_j|$; $\alpha_q \leftarrow B^{-1} \bar{A}_{:q}$
17: $r \leftarrow$ ratio test on $x_B$ over the stable pivots of $\alpha_q$; $\rho_r \leftarrow B^{-T} e_r$; $\alpha_r^T \leftarrow \rho_r^T \bar{A}$
18: **else**
19: **return** $\lambda$ ▷ optimal; $\star$ a solver refactorizes and verifies first
20: $x_B \leftarrow x_B - \theta_p \alpha_q$; $\lambda \leftarrow \lambda + \theta_d \rho_r$; $d \leftarrow d - \theta_d \alpha_r$ ▷ step lengths of the ratio test
21: exchange $r$ and $q$ in $B$; update its LU factors ▷ Forrest–Tomlin
22: **if** 50 pivots since the last factorization **then** refactorize $B$
23: **return** $\lambda$ ▷ iteration limit; $\lambda$ still certifies

*midió: banco de la Sección 5.1?*]. SC therefore uses the primal warm start in the optimizer, where the dual one is 4% slower, and the dual one on systems of equations, where both cost about the same; with `compo`, where neither wins on every system, it alternates the two on the first nodes and keeps the one with the lower measured CPU time per LP, trying the other again every 25 nodes (so the choice, unlike the rest of SC, depends on timing).

### 4.2 Simplex iterations

We follow the bounded-variable simplex of [16]. Each iteration exchanges one basic and one nonbasic variable using the pivot column $\alpha_q = B^{-1} \bar{A}_{:q}$ of the entering variable (FTRAN), the pivot row $\alpha_r^T = \rho_r^T \bar{A}$ with $B^T \rho_r = e_r$ (BTRAN), and updates $x_B \leftarrow x_B - \theta_p \alpha_q$, $\lambda \leftarrow \lambda + \theta_d \rho_r$ and $d \leftarrow d - \theta_d \alpha_r$ (line 20 of Algorithm 2), so that $\lambda$, which the certificate needs, is always at hand.

**Dual steps.** In a dual iteration (lines 10–13 of Algorithm 2) the basis is dual feasible and some basic variable violates its bounds. The leaving variable is the basic variable with the largest violation. The entering variable comes from the ratio test over the pivot row: among the nonbasic $z_j$ whose reduced cost would change sign in the wrong direction, the one with the smallest $|d_j / \alpha_{rj}|$ keeps the basis dual feasible; among the candidates within a relative band of that ratio we take

the largest $|\alpha_{rj}|$ (Harris' rule), which avoids tiny pivots. If no entering variable exists, the LP is infeasible and $\rho_r$ is a Farkas ray (line 13).

**Rules left out.** We also implemented the standard refinements of the dual method [16]: devex pricing [13] for the leaving variable and the long-step (bound flipping) ratio test [18]. On average they did not pay (Section 6), and SC uses the textbook rules above: on the small LPs of a pass, the choice of pivots matters less than the work that surrounds them.

**Primal steps, tolerances.** Primal steps (lines 15–17) enter the nonbasic variable with the largest reduced cost of the wrong sign (Dantzig's rule; with `compo`, whose primal warm starts are longer, devex weights); pivots below a stability threshold are filtered out before choosing the leaving variable, since without this filter a rejected pivot was indistinguishable from optimality. The feasibility tolerances are $10^{-9}$, relative to the value tested, and $10^{-6}$ with `compo` on systems of equations, whose nearly parallel rows make the LPs degenerate. A tolerance is a matter of efficiency, not of rigor: too wide, it weakens the contraction; too tight, the warm-started simplex reports infeasibility on boxes that are not empty, the ray fails the residual test, and SC solves that LP again from the slack basis.

**Factorization.** SC keeps a sparse LU factorization of $B$, computed by the factorization code of SoPlex and updated after each pivot by the Forrest–Tomlin method [10], which reuses the FTRAN of the entering column. It refactorizes every 50 pivots (line 22) and recomputes $x_B$ every 32, to limit drift. With one slack per row, the basis of SC has dimension $m$, the number of rows: SC works in what SoPlex calls the column representation.

### 4.3 Certification

When an LP terminates, the certificate $\gamma = \bar{A}^T\lambda$ is computed once in outward-rounded interval arithmetic, over the rows with $\lambda_i \neq 0$, and $\mathbf{C}_k$ and $\mathbf{R}$ follow in $O(n+m)$. When a dual iteration finds a violated basic variable and no entering variable in its row $r$, $\lambda = \rho_r$ is a Farkas ray, and the residual test turns it into a proof of emptiness. Certifying a bound costs the same as the Neumaier–Shcherbina post-processing of an external solver's dual (1.3–3.1 $\mu$s per LP on three optimization problems); the difference between the approaches is not the cost of a certificate but how $\lambda$ is produced: by a call that treats each LP as a new problem, or by one simplex that serves the whole pass.

## 5 Experimental evaluation

**A ladder of configurations.** All configurations run inside the *same* contractor: the same pass over the $2n$ bounds, with the same skipping rule, the same certificate $\gamma = \bar{A}^T\lambda$ applied to whatever multipliers the engine returns (its dual solution, or its Farkas ray if it reports infeasibility), and the same policy when an LP ends without an optimum: the multipliers where it stopped certify a weaker bound, and the pass continues (Section 3.2). The configurations differ in one step at a time:

R0 *SoPlex as Ibex uses it*: SoPlex 4.0.2 through the LP interface of Ibex, which stores the $n$ variable bounds as rows, with presolving on, the tolerance $10^{-9}$, the nearest bound first as in the contractor of Ibex, and at most 100 iterations per LP;

R1 + *merging twin rows* (Section 3.3);

R2 + *a tuned SoPlex*: its own instance holding the box as column bounds, presolving off, the tolerance $10^{-6}$ for systems of equations and $10^{-9}$ for optimization;[1]

R3 *the bare simplex*: the simplex of SoPlex driven directly through its `SPxSolver` class, the LP kept loaded, only the objective and the column bounds changed between LPs, without the checks, copies and unscaling of the `SoPlex` class; `SPxSolver` itself still refactorizes before declaring an optimum and checks the precision of its solution. It is the cheapest way to use SoPlex. As in R2, the rule of SoPlex chooses the representation of the basis: by columns (dimension $m$), or by rows (dimension $n$) when the relaxation has more than 1.2 rows per variable;

R4 *SC* (Section 4);[2]

R5 *the hybrid*: SC on relaxations of at most 30 rows, the bare simplex beyond.

Each rung is compared with R0. HiGHS 1.15.1 and Gurobi 12.0.3, with one thread, the dual simplex, the basis kept between LPs and no presolve, cost 2.3–4.6 times as much per pass as the tuned SoPlex, so we leave them out of the ladder.

**Setting.** Contractor and solvers are implemented in Ibex 2.8.9, with the affine and `compo` relaxations of Section 3.1. Optimization problems use the affine relaxation, since in preliminary experiments `compo` cost more than it saved with every engine; systems of equations are evaluated with both, since `compo` is the better one on large systems [2] and the affine one keeps most relaxations small. Per node, after merging, the affine relaxation has a median of 5 rows (quartiles 3–9) on optimization problems and 16 (9–30) on systems of equations; `compo` has 56 (36–94) on systems. We call an instance, with a given relaxation, *small* if its merged relaxation has at most 30 rows on average over the nodes, and *large* otherwise; these are the two size classes of the tables. For systems of equations we use the solver composition of Ibex (HC4, ACID, interval Newton when the system is square, and the linear contractor within a fixed point with HC4) with the LSmear bisector, on the 242 systems of the Ibex solver benchmark that some configuration solves within 3600 s, 2 seeds, 60 s per run; for optimization, IbexOpt on the 240 problems of the Ibex optimization benchmark that some configuration solves within 3600 s, 4 seeds, 60 s per run. Three of them, bearing, alkyl and alkylbis, are left out of the means of the solver (Section 6). All runs were made on two Intel Xeon E5-2620 v4 processors (2.1 GHz, 16 cores) with 64 GB of memory, under Linux 6.17, with at most 6 GB per run, eight or fourteen runs at a time (the runs of R0 and R1 inside the search were made at eight; the ratio of R2 to R1 is the same against them as against runs of R2 made at eight); the code is compiled with GCC 13.3 (`-O3`) with the GAOL 4.2.0 interval library. CPU times are process times. The code, the banks of passes and the scripts are public (see the statement of code and data availability).

[1] Each of these choices is the best one for SoPlex on these LPs: presolving each new relaxation costs more than it saves on LPs of a few pivots, $10^{-9}$ makes SoPlex report spurious infeasibility on relaxations with nearly equal rows, and $10^{-6}$ weakens its dual on optimization problems. Turning off scaling, or forcing the row or column representation or the primal algorithm, did not help, and SoPlex 7.1.6 was 1.2–1.9 times slower than 4.0.2 on a sample of eight problems.

[2] On systems with the affine relaxation SC keeps the tolerance $10^{-9}$ while R2 and R3 use $10^{-6}$. With $10^{-6}$ a pass of SC costs the same (0.84 instead of 0.85 times that of the bare simplex, geometric mean over the 36 systems) and contracts 1% less.

Table 1: Time per pass of each rung over that of SoPlex as Ibex uses it (R0) on the bank of passes: quotient of the means of the times normalized per instance (see text). Optimization problems and systems of up to 30 rows use the affine relaxation (one optimization problem has 33 rows); systems above 30 rows use `compo`. Last rows: SC over R3 directly, and instances where SC is cheaper than R3.

| | optimization | systems of equations | |
|---|---|---|---|
| rung | (74) | $\leq 30$ rows (30) | $> 30$ rows (41) |
| R1 + merging twin rows | 0.93 | 0.84 | 0.85 |
| R2 + tuned SoPlex | 0.68 | 0.66 | 0.69 |
| R3 bare simplex | 0.52 | 0.51 | **0.55** |
| R4 SC | **0.39** | **0.42** | 0.79 |
| R5 hybrid | 0.39 | 0.42 | 0.55 |
| R4 / R3 | 0.75 | 0.80 | 1.39 |
| R4 / R3, geometric mean | 0.74 | 0.78 | 1.42 |
| SC cheaper than R3 | 70 / 74 | 25 / 30 | 4 / 41 |

**Rigor.** SC is rigorous by Proposition 3; we also checked it against the LP hull. On four systems, for each of the 2 834 bounds where SC contracted more than the polytope-hull contractor of Ibex on the same box and polytope, we solved the bound LP over the input box with a fresh instance of SoPlex; SC attained this hull in 98% of these bounds and went beyond it in 19, all within the $10^{-9}$ feasibility tolerance of the oracle.

### 5.1 Cost of a pass

To measure the contractor without the noise of the search, we built a bank of passes: for 110 instances, 36 systems of equations (one per family) and 74 optimization problems (one per family and size class, among those on which R0 takes more than 1 s), we recorded the input boxes of 1 000 calls of the contractor spread along a search with SC, with the affine relaxation, and replayed every call with each rung from the same input box and relaxation. Because the affine relaxation of only six systems exceeds 30 rows, a second bank does the same with `compo`, the best relaxation on large systems, for 41 systems whose relaxation has more than 30 rows (31 to 687, median 63; one per family, R0 above 1 s). The 74 optimization problems do not count three that were left out: on pentagon and ex8_2_4, SoPlex as Ibex uses it aborted with an internal exception, and on concon the affine linearization failed for every rung. Two choices of the bank should be kept in mind: its boxes are those visited by a search with SC, although every rung replays them from the same input, and its instances are those on which R0 is slow enough to be measured. Table 1 gives the time per pass of each rung relative to R0. For each instance we normalize both times by the larger one, $t(A)/\max(t(A), t(\mathrm{R0}))$ and $t(\mathrm{R0})/\max(t(A), t(\mathrm{R0}))$, average each over the instances, and report the quotient of the two means; the normalization keeps a few long instances from dominating.

On optimization problems and on systems of up to 30 rows, a pass of SC costs 0.39–0.42 times that of R0, and 25% and 20% less than the bare simplex (SC is cheaper on 70 of the 74 optimization problems and 25 of the 30 systems). The hybrid differs from SC on one optimization problem, of 33 rows. Before SC, merging saves 7–16%, tuning SoPlex 21–27% of what remains, and driving its simplex directly 23–24% more. The median pass on optimization problems takes 264 $\mu$s with R0, 136 with the bare simplex and 88 with SC; on small systems, 199, 109 and 85 $\mu$s (medians weigh

the many fast instances, where SC gains most).

The gain of SC lies in the cost of an LP, not in its pivots. The tuned SoPlex and the bare simplex need a median of 1.8 pivots per LP, SC 2.8, because SoPlex often ends an LP at a vertex that is already optimal for the next bound (Section 6); yet an LP of SC costs 11.6 $\mu$s on optimization problems and 16.3 on small systems, against 16.2 and 21.6 for the bare simplex. What SC does not do is what makes the difference. We measured it against the tuned SoPlex (R2), whose calls perform all of that work: re-inserting into SC, at every LP, the work of a call of the tuned SoPlex, on separate vectors so that the contraction does not change, brings a pass of SC from 0.55–0.60 to 0.88–0.92 times that of R2 (geometric means on 100 of the 104 instances; the emulation failed on four): refactorizing at every LP accounts for 49–56% of the gap (in logarithm), verifying the solution for 22–23%, and recomputing the vectors and copying the solution for 7–8%. Against the bare simplex (R3) the picture changes, since R3 already skips the checks and copies of the `SoPlex` class. Its own refactorization before an optimum and precision check are not what separates it from SC: with two switches added to `SPxSolver` that skip them, the factorizations per LP drop from 0.79 to 0.10 on ex7_2_3, yet a pass of the bare simplex becomes only 7% cheaper on the optimization problems and 4% on the systems, with the same contraction, and SC remains 22% and 12% cheaper (geometric means; it wins on 69 of the 74 problems and 22 of the 36 systems). On bases of 5 to 16 rows a factorization is cheap. What remains is the cost of a call of `SPxSolver::solve` as such: it prepares every LP as a new problem, rebuilding its primal and dual vectors and its pricing and ratio-test vectors from the factorization (4–25% of its time in profiles of four problems), and runs a per-pivot bookkeeping built for general LPs. SC is built for the $2n$ LPs of a pass: between two LPs it recomputes the multipliers and reduced costs for the new objective, and the basic values when the dual warm start relocates nonbasic variables, and keeps its factorization.

**Merging pays more with SC.** Ordered as a ladder, each step is measured on the engine of its rung, and merging is measured on R0, the most expensive one. On the same bank, merging saves 7% of a pass of R0 on optimization problems and 16% on systems of equations, but 21% and 44–47% of a pass of SC. A call of SoPlex carries a fixed cost that does not depend on the rows; SC has almost none, so its time follows the size of the LP, and halving the rows of a square system nearly halves it. Merging is a simple preprocessing, not a contribution of this paper, but it is worth most with an engine that has removed the fixed cost of a call.

**Same contraction.** All rungs empty the same boxes and contract alike: the sum of the widths of the box shrinks by 25.5% per call on average on optimization problems and by 18.6–18.7% on small systems for R1 to R4, and within 1% on all but 6–9 of the 110 instances. The exception is R0, which ends 8.5% of the LPs of small systems at its limit of 100 iterations (0.9% after merging; the tuned SoPlex, with the same limit, almost none); the multipliers where it stops certify a weaker bound, and R0 contracts by 18.2% (on Geneig-icse, the extreme case, by 23% against 34%). On the `compo` bank the LPs are larger and more degenerate, and with the tolerance of $10^{-6}$ the certified bounds of the engines differ slightly more: SC contracts by 15.6%, the tuned SoPlex by 15.9% and the bare simplex by 16.1%, while the limit of R0 ends 5% of the LPs without an optimum.

**Large relaxations.** On the systems whose relaxation exceeds 30 rows the order of the last two rungs is reversed (Figure 1): on the `compo` bank a pass of SC costs 1.39 times that of the bare simplex (1.42 in geometric mean) and is cheaper on only 4 of the 41 systems, and on the six affine ones it costs 1.49 times (geometric mean) and loses on all. Merging and tuning SoPlex keep their weight; the hybrid hands these relaxations to the bare simplex.

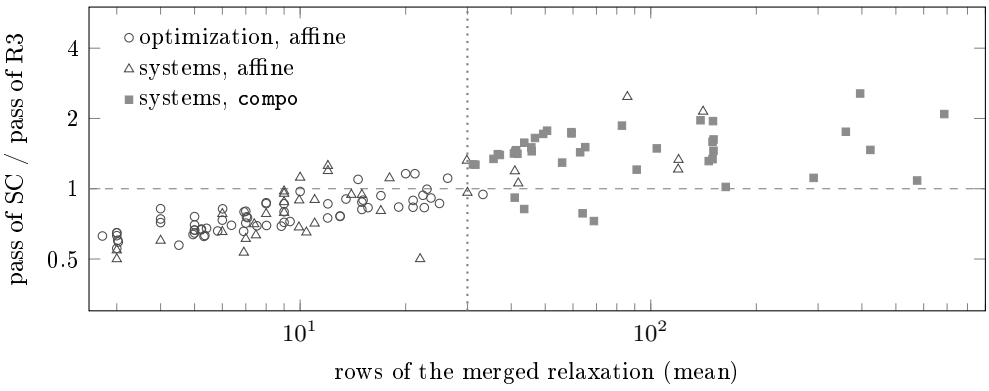



Figure 1: Time per pass of SC over that of the bare simplex (R3) on the two banks, one point per instance, against the size of the relaxation. Below the dashed line SC is cheaper; the dotted line is the threshold of the hybrid.

**Why SC is slower there.** What SC saves is a fixed cost per LP, and on these relaxations an LP needs a dozen pivots instead of two or three, so the saving no longer weighs. What remains is the cost of the pivots, and three different things happen there.

*The representation of the basis* is the main one. A vertex of the relaxation can be described by a basis of dimension $m$, with one basic column per row, or by a basis of dimension $n$, the $n$ rows and bounds that are tight. SC, with one slack per row, always works with the first. SoPlex chooses: by columns when the relaxation has at most 1.2 rows per variable, by rows otherwise. With `compo` the rows outnumber the variables several times (147 rows for 35 variables on Fourbar-icse), so the bare simplex factorizes and solves with a basis of dimension $n$ where SC uses one of dimension $m$: with almost the same pivots (1.05 times), SC spends there three times as long in the triangular solves of the entering column. To measure how much of the gap this explains, we forced the bare simplex to the column representation on the `compo` bank. It becomes 1.35 times slower (geometric mean), with the same contraction, and a pass of SC then costs 1.04 times that of the bare simplex instead of 1.41 in the same runs, and is cheaper on 18 of the 41 systems; on the systems without the two difficulties below (the Redeco and Eco families, Katsura-30, Dietmaier, Directkin, Butcher) the two are within 10% of each other. The representation thus accounts for almost 90% of the gap, in logarithm. On the bank of the affine relaxation, where merging leaves about as many rows as variables, SoPlex already works by columns, and forcing it to rows makes it slower (a pass of SC goes from 0.85 to 0.79 times that of the bare simplex).

*Stalling* remains on a few systems: on Katsurabis-14, 17 of the 16 562 LPs of SC, degenerate ones, pivot until the limit of 100 iterations, while the bare simplex reaches an optimum on all of its LPs; the primal steps of SC have no protection against stalling.

*The systems with band structure* (Bratu, BroydenTri, BroydenBanded, DiscreteBoundary) are the third case: there SC makes several times the pivots of the bare simplex, up to eleven times on the six affine systems above 30 rows, where both engines work by columns. Devex pricing and the long-step ratio test (Section 4.2) help on them with the affine relaxation: they bring a pass of SC from 0.71 to 0.55 times that of R0 on those six systems (the bare simplex: 0.47), but leave it at 1.51 times the bare simplex on the `compo` bank (geometric mean), and change little on the smaller relaxations. We have no explanation for these systems beyond the pivoting rules of SoPlex, its automatic pricer with the bound-flipping ratio test.

Table 2: CPU time of each rung over that of SoPlex as Ibex uses it (R0), inside the search, one instance per family and size class, over the runs where one of the two configurations takes more than 1 s (resp. 5 s); in parentheses, the families of each class before that filter. On the classes up to 30 rows the hybrid is not reported: it runs SC on almost every node. Systems of equations pool the affine and `compo` relaxations. The search trees of R1 to R5 agree within 3%.

| | optimization | | systems of equations | | | |
|---|---|---|---|---|---|---|
| | $\leq 30$ rows (189) | | $\leq 30$ rows (65) | | $> 30$ rows (52) | |
| rung | $> 1$ s | $> 5$ s | $> 1$ s | $> 5$ s | $> 1$ s | $> 5$ s |
| R1 + merging twin rows | 0.97 | 0.95 | 0.96 | 0.95 | 0.91 | 0.89 |
| R2 + tuned SoPlex | 0.90 | 0.90 | 0.89 | 0.87 | 0.85 | 0.82 |
| R3 bare simplex | 0.84 | 0.85 | 0.85 | 0.84 | 0.79 | 0.76 |
| R4 SC | **0.78** | **0.79** | **0.83** | **0.83** | 0.91 | 0.87 |
| R5 hybrid | – | – | – | – | **0.77** | **0.75** |

## 5.2 The whole solver

Table 2 runs the same ladder inside the search. To weight families equally, we keep one instance per family and size class, the one with the largest relaxation; families are the prefix of the name on systems of equations, while on optimization problems every problem is its own family, except for reformulations of the same problem (`bis`, `-1`). To populate both size classes of the systems, we pool their runs with the affine and `compo` relaxations, each (instance, relaxation) classified by its own size. Times are normalized per run as on the bank; unfinished runs count as 60 s.

The ladder keeps its shape, but the gains are diluted: the pass is only part of the cost of a node, with the linearization, HC4, ACID and interval Newton. On optimization problems the solver takes 21–22% less CPU time with SC than with R0, and 5–8% less than with the bare simplex. On small systems SC and the bare simplex nearly tie (0.83 against 0.84–0.85), because the pass weighs less in the system solver. Counting the calls of the contractor in a search with SC and multiplying them by the cost of a pass on the bank, the pass takes a median of 16% of the CPU on optimization problems (interquartile range 7–26%) but 5% on systems of equations (2–13%), where the linearization, HC4, ACID and interval Newton dominate a node; saving 20% of 5% is about 1%. On large systems SC is 15–16% slower than the bare simplex, and the hybrid, which runs the bare simplex there, matches it (0.98). Only three optimization families exceed 30 rows (33–49 rows); SC remains the best on them (0.82 against 0.97 for the bare simplex), so the hybrid could keep SC on optimization problems at any size. Devex pricing and the long-step ratio test change these ratios by at most 0.01 up to 30 rows, and make SC 4–6% slower on large systems.

**The threshold of the hybrid.** From the runs of every rung we evaluated offline the hybrid that takes, for each (instance, relaxation), SC if its mean relaxation has at most $T$ rows and the bare simplex otherwise. Against the bare simplex, SC gains below 15 rows (0.86–0.92 on optimization problems, 0.95–0.98 on systems), ties from 15 to 30, and loses from 40 rows on (1.05 at 40–50 rows, 1.15 beyond 120 on systems). The total time of the hybrid is flat for $T$ between 15 and 30, and we use $T = 30$; the contractor applies it per node, with the size of the relaxation of the previous node. Run inside the search on the systems of equations with `compo` (two seeds), the hybrid with $T = 15$, 20 and 30 costs 0.98–1.00 times the bare simplex on the small systems and 0.98 on the large ones, where SC alone costs 1.15: the choice per node loses nothing, and the threshold does not matter

inside this range.

## 6 Discussion

**Why SC wins.** SC does not contract with a better polytope: every rung solves the same relaxation and feeds the same certificate, and from R1 on they contract alike. Its advantage is the cost of an LP. The ladder separates two parts of the cost of a call of SoPlex as Ibex makes it: the interface and its options (R1 to R3), which a careful integration of SoPlex removes, and what the simplex of SoPlex itself does at every LP, which only a simplex dedicated to the pass can omit. SC pivots more than SoPlex, and each LP still costs it less. Against the calls of the `SoPlex` class, the emulation of Section 5.1 attributes most of the gap to the refactorization and the verification that SC omits. Against `SPxSolver` driven directly, removing those two from SoPlex recovers little: what SC saves is the preparation of each LP as a new problem, which a solver built for one LP repeats at every call. That SC needs no phase 1 when the objective or the box changes, and keeps one basis for the whole pass, is what lets it skip that preparation: the certificate makes any basis usable, so between two LPs only what the new objective changes has to be recomputed. On small relaxations, where an LP needs two or three pivots, that saves 20–25% of a pass of the bare simplex.

**Where it loses.** The saving is a fixed cost per LP, so its weight falls as the pivots per LP grow, and beyond 30 rows they dominate. There the cost of a pivot decides, and SoPlex has an option that SC lacks: when the rows outnumber the variables, as with `compo`, it represents the basis by rows, with the dimension of the variables instead of that of the rows. Against SoPlex in the column representation SC is within 4% (Section 5.1); the rest comes from a few degenerate LPs on which its primal steps stall, and from systems with band structure on which it pivots more. The homogeneous form with one slack per row, which makes the certificate and the warm starts simple, is what ties SC to the column representation; a self-certifying simplex in the row representation would keep the certificate, which needs only the multipliers, and is the natural next step. Meanwhile the hybrid uses each engine where it is better; on systems of equations, whose best relaxation is `compo`, this is what makes the redesigned contractor pay. On small relaxations one difference remains unexplained: SoPlex ends 13–40% of its LPs on optimization problems at a vertex that is already optimal for the next bound, and needs fewer pivots than SC (1.8 against 2.8 per LP); neither a tie-break toward the corners of the box inside the optimal face nor a short walk inside that face made SC end at such vertices.

**What did not pay.** The refinements of the dual method left out of SC (Section 4.2), devex pricing and the long-step ratio test, help on some relaxations and hurt on others: on the bank they make a pass 1.3–1.5 times faster on three of the band systems with the affine relaxation, and up to 2.1 times slower on the same systems with `compo`, and on average they leave the time of the search unchanged (Section 5.2). Several ideas that use the certificate or the tableau beyond the pass did not pay on the whole benchmark either. Certificates of an ancestor box remain valid (Proposition 3) but rarely prune what the node does not prune anyway. Truncating each LP when the certified bound of its current basis (Proposition 5) is close to what the LP can still gain saves pivots ($\times 0.84$) but weakens the contraction (nodes $\times 1.39$–$1.60$). Propagating with HC4 in the middle of a pass, cuts at the LP optima, and a Lagrangian bound improved by monotonicity reduce the nodes or tighten bounds, but cost more than they save. Inheriting the basis of the parent node cuts the pivots of the first LP of a node by a factor of three or more, but the first LP is a small part of the node. Earlier versions of SC kept the whole tableau explicitly and perturbed the zero reduced

costs of the highly dual-degenerate bound problems [16]; with the other components in place both made SC slower. Doing more work inside a node did not change the search tree; what changed it was what is computed: the stability filter of the ratio test, the Farkas ray of the infeasible row, the tolerance.

**Limits.** SC removes the external LP solver from the contractor, not from the optimizer: the upper-bounding of IbexOpt still solves its LPs with SoPlex. On problems with very narrow equalities the upper-bounding becomes the bottleneck: on bearing, whose eight equalities are relaxed to $|h| \leq 10^{-8}$, feasible points are found only in boxes of width $10^{-4}$ to $10^{-3}$, and the search tree changes by two orders of magnitude between seeds; this is why bearing, alkyl and alkylbis are left out of the means of the solver. Relaxations of several hundred rows are still expensive for every engine: the systems of 400 to 1 000 variables of the benchmark exceed the limits of 60 s and 6 GB with all of them. And the gain of SC on the whole system solver is small, because the pass is a small part of its nodes (5% of the CPU in the median). One use of the certificate is left open: when the warm-started simplex declares an infeasibility that the ray does not certify, SC solves the LP again from the slack basis (Section 4.2), but the multipliers of the basis where it stopped, moved along the ray as long as the bound improves, already give a certified bound without that second LP.

## 7 Conclusion

A polytope-hull contractor solves $2n$ small, closely related LPs per node and certifies each bound with the Neumaier–Shcherbina bound, which is valid for any multipliers. A general-purpose LP solver cannot take advantage of this: it treats each LP as a new problem. A simplex dedicated to the pass solves the $2n$ problems in sequence on one basis, updating only what changes from one LP to the next; the certificate makes it simple, since it never verifies or refactorizes before an optimum and needs no phase 1. Switching off the refactorization and the checks in SoPlex recovers little of the difference: the gain is in not preparing each LP as a new problem. Measured as a ladder inside the same contractor, from the same boxes and relaxations, a pass of this simplex costs 0.39–0.42 times that of SoPlex as Ibex uses it on relaxations of up to 30 rows, and 20–25% less than the simplex of SoPlex driven directly, with the same contraction. Inside the search, the optimizer takes 21–22% less CPU time than with SoPlex as Ibex uses it, and 5–8% less than with the bare simplex. On larger relaxations the pivots dominate, and when the rows outnumber the variables SoPlex makes them cheaper by representing the basis by rows: against SoPlex in the column representation, the one of SC, a pass of SC is within 4%. A hybrid that chooses the engine by the size of the relaxation keeps the best of both. A self-certifying simplex in the row representation is the natural next step.

## Code and data availability

The code and data used in the experiments are archived at Zenodo (**[TODO:** *DOI*]); they are the branch `dfb-sc-paper` of `https://github.com/rilianx/ibex-lib`. SC and the contractor are in `examples/dfb`, and a standalone version of SC that builds against an installed Ibex is in `examples/sc_plugin`; `results/escalera_2026-11-06` holds the banks of passes (input boxes and relaxations), the raw results of every run, the scripts that ran them and those that produce the tables, with instructions to rebuild the binaries. The fix to the polytope-hull contractor of Ibex used by every configuration was submitted upstream (`https://github.com/ibex-team/ibex-lib/pull/579`).

## Use of generative AI

The authors used Claude (Anthropic), through the Claude Code assistant, to help implement the contractor and the simplex, to write the scripts that run and analyze the experiments, and to draft and edit the text. The authors directed the work, checked the code, the results and the text, and take full responsibility for the content of the paper.

## References


[1] David L. Applegate, William Cook, Sanjeeb Dash, and Daniel G. Espinoza. Exact solutions to linear programming problems. *Operations Research Letters*, 35(6):693–699, 2007.

[2] Ignacio Araya, Frédéric Messine, Jordan Ninin, and Gilles Trombettoni. Hybridizing two linear relaxation techniques in interval-based solvers. *Journal of Global Optimization*, 91(3):437–456, 2025.

[3] Ignacio Araya, Gilles Trombettoni, and Bertrand Neveu. A contractor based on convex interval Taylor. In *Integration of AI and OR Techniques in Constraint Programming (CPAIOR 2012)*, Lecture Notes in Computer Science, pages 1–16, 2012.

[4] Ali Baharev, Tobias Achterberg, and Endre Rév. Computation of an extractive distillation column with affine arithmetic. *AIChE Journal*, 55(7):1695–1704, 2009.

[5] Pietro Belotti, Jon Lee, Leo Liberti, François Margot, and Andreas Wächter. Branching and bounds tightening techniques for non-convex MINLP. *Optimization Methods and Software*, 24(4-5):597–634, 2009.

[6] Frédéric Benhamou, Frédéric Goualard, Laurent Granvilliers, and Jean-François Puget. Revising hull and box consistency. In *Proceedings of the International Conference on Logic Programming (ICLP)*, pages 230–244, 1999.

[7] Gilles Chabert and Luc Jaulin. Contractor programming. *Artificial Intelligence*, 173(11):1079–1100, 2009.

[8] William Cook, Thorsten Koch, Daniel E. Steffy, and Kati Wolter. A hybrid branch-and-bound approach for exact rational mixed-integer programming. *Mathematical Programming Computation*, 5(3):305–344, 2013.

[9] Ferenc Domes and Arnold Neumaier. Rigorous filtering using linear relaxations. *Journal of Global Optimization*, 53(3):441–473, 2012.

[10] John J. H. Forrest and John A. Tomlin. Updated triangular factors of the basis to maintain sparsity in the product form simplex method. *Mathematical Programming*, 2:263–278, 1972.

[11] Ambros M. Gleixner, Timo Berthold, Benjamin Müller, and Stefan Weltge. Three enhancements for optimization-based bound tightening. *Journal of Global Optimization*, 67(4):731–757, 2017.

[12] Ambros M. Gleixner, Daniel E. Steffy, and Kati Wolter. Iterative refinement for linear programming. *INFORMS Journal on Computing*, 28(3):449–464, 2016.

[13] Paula M. J. Harris. Pivot selection methods of the Devex LP code. *Mathematical Programming*, 5(1):1–28, 1973.

[14] Christian Jansson. Rigorous lower and upper bounds in linear programming. *SIAM Journal on Optimization*, 14(3):914–935, 2004.

[15] Christian Keil and Christian Jansson. Computational experience with rigorous error bounds for the Netlib linear programming library. *Reliable Computing*, 12(4):303–321, 2006.

[16] Achim Koberstein. *The dual simplex method, techniques for a fast and stable implementation.* PhD thesis, Universität Paderborn, 2005. SIN VERIFICAR.

[17] Yahia Lebbah, Claude Michel, and Michel Rueher. A rigorous global filtering algorithm for quadratic constraints. *Constraints*, 10:47–65, 2005. SIN VERIFICAR (tomado de las referencias de Domes y Neumaier 2012).

[18] István Maros. A generalized dual phase-2 simplex algorithm. *European Journal of Operational Research*, 149(1):1–16, 2003.

[19] Arnold Neumaier. *Interval Methods for Systems of Equations.* Cambridge University Press, 1990.

[20] Arnold Neumaier. Complete search in continuous global optimization and constraint satisfaction. In *Acta Numerica 2004*, pages 271–369. Cambridge University Press, 2004. SIN VERIFICAR (tomado de las referencias de Domes y Neumaier 2012).

[21] Arnold Neumaier and Oleg Shcherbina. Safe bounds in linear and mixed-integer linear programming. *Mathematical Programming*, 99(2):283–296, 2004.

[22] Bertrand Neveu, Gilles Trombettoni, and Ignacio Araya. Adaptive constructive interval disjunction: algorithms and experiments. *Constraints*, 20(4):452–467, 2015.

[23] Jordan Ninin, Frédéric Messine, and Pierre Hansen. A reliable affine relaxation method for global optimization. *4OR*, 13(3):247–277, 2015.

[24] Victor Reyes and Ignacio Araya. Non-convex optimization: Using preconditioning matrices for optimally improving variable bounds in linear relaxations. *Mathematics*, 11(16):3549, 2023.

[25] Hong S. Ryoo and Nikolaos V. Sahinidis. A branch-and-reduce approach to global optimization. *Journal of Global Optimization*, 8(2):107–138, 1996.

[26] Roland Wunderling. *Paralleler und objektorientierter Simplex-Algorithmus.* PhD thesis, Technische Universität Berlin, 1996.

[27] Kiyotaka Yamamura and T. Fujioka. Finding all solutions of systems of nonlinear equations using the dual simplex method. In *Proc. 2001 Int. Symp. Nonlinear Theory and Its Applications (NOLTA)*, pages 219–222, 2001. SIN VERIFICAR (tomado de las referencias de Yamamura y Tanaka 2006).

[28] Kiyotaka Yamamura and Katsumasa Tanaka. Finding all solutions of weakly nonlinear equations using the dual simplex method. *Electronics and Communications in Japan (Part III: Fundamental Electronic Science)*, 89(7):1–7, 2006.